SceneForge: Failure-Aware Adaptive Orchestration for Multi-Stage AI Video Pipelines

1st Shridhi Gupta
Faculty of Engineering)
Teerthanker Mahaveer University
Moradabad, Uttar Pradesh, India
guptashridhi11@gmail.com

2nd Nripesh Kumar
Faculty of Engineering
Teerthanker Mahaveer University
Moradabad, Uttar Pradesh, India
@gmail.com

3nd Ajay Singh
Faculty of Engineering
Teerthanker Mahaveer University
Moradabad, Uttar Pradesh, India
@gmail.com

***Abstract*—Recent advances in generative artificial intelligence have enabled automated video generation from textual inputs through multi-stage pipelines comprising scene decomposition, image synthesis, clip generation, and final rendering. However, such pipelines are inherently stochastic, resource-intensive, and failure-prone due to model variability, external service dependencies, and long-running distributed execution. Existing systems predominantly rely on static orchestration and naive retry mechanisms, leading to increased computational cost, latency, and inconsistent output quality.**

In this paper, we propose a Failure-Aware Adaptive Orchestration Framework (FA-AOF) for robust and efficient multi-stage AI video generation. The proposed framework introduces three key components: (i) a failure classification module that categorizes errors based on operational and semantic characteristics, (ii) a dynamic strategy selection mechanism that adaptively determines recovery actions such as prompt modification, model switching, or parameter tuning, and (iii) a cost-quality optimization layer that balances resource utilization with output fidelity.

The framework is implemented using a distributed architecture leveraging FastAPI, Celery, and Redis, enabling scalable and fault-tolerant execution of long-running AI workflows. Extensive experiments conducted on script-driven video generation tasks demonstrate that the proposed approach reduces failure rates, improves output consistency, and optimizes computational cost compared to baseline static retry systems.

The proposed method provides a generalized solution for intelligent orchestration in generative AI pipelines and can be extended to other multi-stage AI systems beyond video generation.

Extensive experiments conducted across multiple pipeline executions demonstrate that the proposed approach reduces failure rates, improves output consistency, and optimizes computational cost compared to baseline static retry systems.

***Keywords*—Generative Artificial Intelligence, Text-to-Video Generation, AI Pipelines, Workflow Orchestration, Failure-Aware Systems, Adaptive Retry Mechanisms, Distributed Systems, Fault Tolerance**

I. Introduction

A. Background and Motivation

The rapid evolution of generative artificial intelligence has significantly advanced the automatic creation of multimedia content, including images, audio, and videos, from textual descriptions [31], [32]. Among these, text-to-video generation has emerged as a complex yet promising domain, leveraging deep learning architectures such as generative adversarial networks (GANs) and diffusion models to synthesize temporally coherent video sequences [2], [6], [28], [29]. Early work by Vondrick et al. demonstrated the feasibility of generating videos with dynamic scene representations [1], while recent approaches such as CogVideoX have further improved controllability and realism in text-to-video synthesis [3]. Foundational temporal modeling concepts, such as those introduced by LSTM architectures, continue to influence video sequence generation [34].

Most modern systems adopt a multi-stage pipeline architecture, where a user-provided script is decomposed into scenes, followed by image synthesis, clip generation, and final video rendering [7]. Despite these advancements, building reliable end-to-end systems for video generation remains a challenging task due to the integration of heterogeneous components and long-running processing stages.

B. Challenges in Multi-Stage AI Video Pipelines

Multi-stage AI pipelines for video generation face several critical challenges. First, generative models are inherently stochastic, often producing inconsistent or low-quality outputs across different executions [4], [5]. Second, these pipelines depend on external services and computationally intensive operations, making them prone to failures such as timeouts, resource exhaustion, and service unavailability [15]. Third, the sequential dependency between pipeline stages amplifies errors, where failures in earlier stages propagate to subsequent stages, leading to degraded overall performance.

Furthermore, traditional orchestration systems rely on static retry mechanisms that do not consider the underlying cause of failures. Such approaches lead to inefficient resource utilization, increased latency, and higher computational costs in distributed environments [17], [18].

C. Limitations of Existing Approaches

Existing approaches to workflow orchestration in AI systems primarily focus on scalability, scheduling, and resource management using distributed frameworks such as Apache Spark and Airflow [10], [11]. While these systems provide basic fault tolerance through retry mechanisms and logging,

they lack adaptive intelligence in handling failures, treating all errors uniformly regardless of their context [8].

Recent developments in machine learning pipelines, including TensorFlow Extended (TFX) and similar frameworks, have improved pipeline automation and monitoring [12]. However, these approaches do not incorporate failure-aware adaptive strategies, particularly in the context of generative AI workflows. Additionally, emerging distributed systems and orchestration tools such as Kafka-based pipelines and microservices architectures enable scalable execution but still rely on static control logic [19], [20].

D. Proposed Approach

To address these challenges, we propose a Failure-Aware Adaptive Orchestration Framework (FA-AOF) for multi-stage AI video generation pipelines. The proposed framework introduces a structured failure classification mechanism that categorizes errors based on operational and semantic characteristics, inspired by prior work on failure analysis in machine learning systems [15]. Based on this classification, a dynamic strategy selection module determines appropriate recovery actions, such as prompt modification, model switching, or parameter tuning.

Additionally, the framework incorporates a cost-quality optimization layer that balances computational resource usage with output fidelity. By leveraging distributed system principles and adaptive decision-making, the proposed approach enhances the robustness and efficiency of long-running AI workflows [21], [22].

E. Contributions

The main contributions of this work are summarized as follows:

- A formal framework for failure-aware orchestration in multi-stage generative AI pipelines.
- A novel failure-aware adaptive orchestration mechanism that dynamically optimizes recovery strategies based on contextual failure classification.
- A distributed system implementation for scalable and fault-tolerant video generation.
- An experimental evaluation demonstrating improvements in reliability, cost efficiency, and output quality.

F. Organization of the Paper

The remainder of this paper is organized as follows: Section II reviews related work, Section III formulates the problem, Section IV presents the proposed methodology, Section V describes the system architecture, Section VI outlines the experimental setup, Section VII discusses the results, and Section VIII concludes the paper with future directions.

II. Related Work

Recent advancements in generative artificial intelligence and distributed systems have significantly contributed to the development of automated multimedia generation pipelines. Research efforts span across video generation models, workflow orchestration frameworks, machine learning pipelines, and failure handling mechanisms. However, the integration of these domains into a unified, failure-aware adaptive system for multi-stage video generation remains relatively unexplored. This section reviews the most relevant literature and highlights the research gap addressed in this work.

A. Generative Models for Video Synthesis

Video generation has been an active area of research, with early approaches focusing on learning temporal dynamics from static images. Vondrick et al. introduced one of the first frameworks for generating videos with realistic scene dynamics [1]. This work was enabled by advances in deep convolutional networks for visual recognition [40]. Many early approaches utilized deep convolutional architectures for feature extraction [38]. Subsequent advances built on temporal modeling techniques pioneered by recurrent architectures [34]. Subsequent works leveraged generative adversarial networks (GANs) to improve spatial and temporal coherence in synthesized videos [6], [35].

More recently, diffusion-based models have emerged as a powerful paradigm for high-quality image and video generation. Ho et al. proposed denoising diffusion probabilistic models, which have been extended to video synthesis tasks [28]. The training of such models typically relies on adaptive optimization algorithms like Adam [41]. Rombach et al. introduced latent diffusion models to improve computational efficiency while maintaining high fidelity [29], [30]. Such models are typically pre-trained on large-scale datasets like ImageNet [39]. Contemporary systems such as CogVideoX utilize transformer-based architectures to enhance controllability and scalability in text-to-video generation [2], [3], [27]. Surveys by Zhou et al. and Wang et al. provide comprehensive overviews of recent advancements in generative AI for video synthesis [4], [5].

B. AI Pipelines and Workflow Orchestration

The increasing complexity of AI systems has led to the adoption of pipeline-based architectures for managing multi-stage workflows. Frameworks such as Apache Spark and MapReduce enable large-scale data processing through distributed computation [9], [10], while workflow orchestration tools like Apache Airflow provide scheduling and dependency management capabilities [11], [30].

Recent studies have explored the application of such frameworks to AI pipelines. Kazlaris et al. demonstrated the use of Airflow for orchestrating scalable AI workflows [8], while Trajkovik et al. proposed an AI-powered pipeline for automated video generation from textual inputs [7]. Additionally, TensorFlow Extended (TFX) provides end-to-end pipeline support for machine learning workflows, including data validation and model deployment [12]. Despite these advancements, existing orchestration systems primarily focus on task execution and lack adaptive mechanisms for handling failures intelligently.

C. Machine Learning Systems and MLOps

The engineering of reliable machine learning systems has gained significant attention in recent years. Sculley et al.

highlighted the concept of hidden technical debt in machine learning systems, emphasizing the complexity of maintaining production-grade pipelines [17]. Amershi et al. further explored software engineering practices for machine learning, identifying challenges in system design, monitoring, and scalability [18].

Modern MLOps frameworks aim to address these challenges by integrating continuous development, testing, and deployment pipelines. Polyzotis et al. proposed data validation techniques for improving pipeline reliability [16], while large-scale systems such as TensorFlow and PyTorch provide foundational infrastructure for building and deploying AI models [13], [36]. However, these approaches focus primarily on model lifecycle management rather than adaptive orchestration in failure-prone environments.

D. Failure Handling and Fault Tolerance in AI Systems

Failure handling is a critical aspect of distributed AI systems, particularly for long-running workflows. Chen et al. analyzed failure modes in machine learning pipelines, identifying common issues such as data inconsistencies, resource failures, and model degradation [15]. Traditional distributed systems employ retry mechanisms and fault tolerance strategies, as seen in platforms such as Kafka and microservices-based architectures [19], [20].

Emerging orchestration frameworks, including durable execution platforms, aim to improve reliability through state persistence and recovery mechanisms [21]. Additionally, large-scale cluster management systems such as Borg and Kubernetes provide resource allocation and fault tolerance capabilities [22], [23]. However, these systems do not incorporate semantic understanding of failures and lack adaptive decision-making for optimizing recovery strategies.

E. Research Gap

From the literature, it is evident that significant progress has been made in generative video models, distributed AI pipelines, and fault-tolerant systems. However, existing approaches largely treat failures in a uniform manner and rely on static retry policies. There is a lack of integrated frameworks that combine failure-aware analysis with adaptive orchestration in multi-stage generative AI pipelines.

To address this gap, the present work proposes a Failure-Aware Adaptive Orchestration Framework (FA-AOF) that introduces intelligent failure classification and dynamic strategy selection mechanisms, enabling efficient and robust execution of video generation workflows.

III. Problem Formulation

Multi-stage AI video generation can be modeled as a sequential pipeline of dependent tasks, where each stage transforms intermediate representations into higher-level outputs. Let the pipeline consist of N ordered stages:

$$P = \{T_1, T_2, \ldots, T_N\} \tag{1}$$

where each task T_i represents a processing stage such as scene parsing, image generation, clip synthesis, or final rendering [7]. The output of each task serves as the input to the subsequent task, forming a directed acyclic graph (DAG) structure commonly used in workflow orchestration systems [8], [11].

Input processing for text-based tasks often leverages established NLP libraries [42], [43].

A. Task Execution Model

Each task T_i is defined as a tuple:

$$T_i = (I_i, O_i, \theta_i, C_i) \tag{2}$$

where I_i is the input, O_i is the output, θ_i represents execution parameters (e.g., model type, prompt configuration), and C_i denotes the computational cost associated with executing the task. The execution of T_i produces an output O_i with a quality score Q_i, which can be estimated using evaluation metrics such as similarity scores or perceptual quality measures [26].

B. Failure Modeling

Due to the stochastic nature of generative models and system-level uncertainties, each task T_i is associated with a probability of failure. Let F_i denote the failure event for task T_i, defined as:

$$F_i = \begin{cases} 1, & \text{if task } T_i \text{ fails} \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

Failures may arise from multiple sources, including model instability, API errors, and resource constraints [15]. We categorize failures into a set of types:

$$\mathcal{F} = \{f_{timeout}, f_{api}, f_{quality}, f_{resource}\} \tag{4}$$

Each failure type provides contextual information that can be used to guide recovery strategies.

C. Static Retry Limitation

Traditional orchestration systems apply a static retry mechanism, where a failed task is re-executed without modifying its parameters:

$$T_i^{(k+1)} = T_i^{(k)} \tag{5}$$

where k denotes the retry attempt. Such approaches ignore failure context and often lead to repeated failures, increased latency, and higher computational cost [17], [18].

D. Adaptive Orchestration Objective

To overcome these limitations, we model the orchestration process as a decision-making problem. At each failure event, the system selects an action $a \in \mathcal{A}$ from a predefined action space:

$$\mathcal{A} = \{\text{retry}, \text{modify\_prompt}, \text{switch\_model}, \text{adjust\_parameters}\} \tag{6}$$

The state of the system at time t is defined as:

$$S_t = (T_i, F_i, Q_i, C_i, k) \tag{7}$$

The objective is to determine an optimal policy $\pi(S_t)$ that minimizes the expected cost while maximizing output quality and success rate:

$$\min_{\pi} \mathbb{E}\left[\sum_{i=1}^{N} C_i\right] \quad \text{subject to} \quad \max_{\pi} \mathbb{E}\left[\sum_{i=1}^{N} Q_i\right] \tag{8}$$

This formulation aligns with optimization principles in distributed systems and reinforcement learning-based decision-making [14], [22]. Adaptive parameter selection in such frameworks is informed by hyperparameter optimization strategies [37].

E. Problem Statement

Given a multi-stage AI pipeline P with stochastic task behavior and failure-prone execution, the goal is to design an adaptive orchestration framework that:

- Identifies and classifies failures based on contextual information,
- Dynamically selects recovery strategies for each task,
- Minimizes overall computational cost and latency,
- Maximizes output quality and pipeline success rate.

The proposed framework addresses this problem by integrating failure-aware analysis with adaptive decision-making in distributed AI workflows.

IV. Proposed Method

In this section, we present the proposed Failure-Aware Adaptive Orchestration Framework (FA-AOF) for multi-stage AI video generation pipelines. The framework is designed to enhance robustness and efficiency by integrating failure classification, adaptive decision-making, and cost-quality optimization into the orchestration process. Unlike traditional static retry mechanisms, the proposed method dynamically adapts execution strategies based on task state and failure context.

A. Framework Overview

The proposed framework models the pipeline as a sequence of interdependent tasks, where each task execution is monitored and evaluated. Upon failure, the system invokes a decision engine that determines the most suitable recovery action. This design is inspired by distributed workflow orchestration systems and adaptive control mechanisms in machine learning pipelines [8], [12].

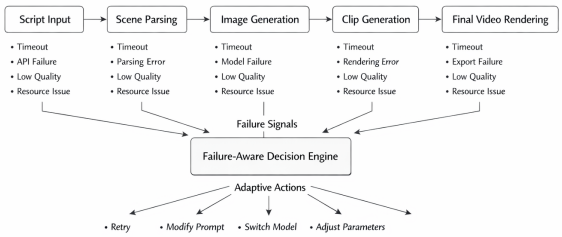

Fig. 1. Failure-aware adaptive workflow for multi-stage AI video generation, illustrating dynamic recovery strategies at each pipeline stage.

The overall workflow consists of three key components:

- Failure Classification Module
- Adaptive Strategy Selection Module
- Cost-Quality Optimization Layer

B. Failure Classification Module

To enable intelligent recovery, failures are first categorized based on their underlying causes. Given a failure event F_i, the classification function $\phi(F_i)$ maps it to a predefined failure category:

$$\phi(F_i) \to f_j \in \mathcal{F} \tag{9}$$

where $\mathcal{F} = \{f_{timeout}, f_{api}, f_{quality}, f_{resource}\}$ as defined earlier. This classification is based on error logs, response times, and output evaluation metrics, similar to failure analysis approaches in machine learning systems [15].

For example:

- Timeout errors $\to f_{timeout}$
- API failures $\to f_{api}$
- Low-quality outputs $\to f_{quality}$
- Resource exhaustion $\to f_{resource}$

This categorization enables context-aware decision-making instead of uniform retry policies.

C. Adaptive Strategy Selection

Once a failure type is identified, the system selects an appropriate recovery action from the action space $\mathcal{A}$. The decision function $\pi(S_t)$ maps the current system state S_t to an action:

$$\pi(S_t) : S_t \to a \in \mathcal{A} \tag{10}$$

where $\mathcal{A} = \{\text{retry}, \text{modify\_prompt}, \text{switch\_model}, \text{adjust\_parameters}\}$.

The selection policy is designed using rule-based heuristics combined with optimization principles inspired by reinforcement learning strategies [14]. For instance:

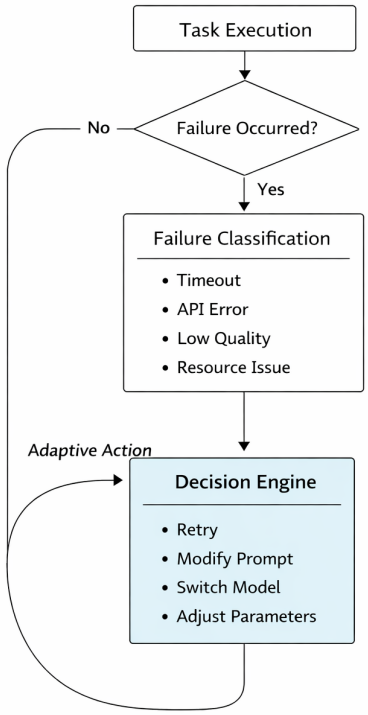

Fig. 2. Failure-aware decision flow illustrating classification of failures and adaptive strategy selection in the proposed framework.

- If $f_{timeout}$, reduce resolution or retry with adjusted parameters.
- If f_{api}, switch to an alternative model or service.
- If $f_{quality}$, modify prompt or regenerate with different parameters.
- If $f_{resource}$, reschedule task or allocate additional resources.

This adaptive mechanism improves success rates and avoids redundant retries.

D. Cost-Quality Optimization Layer

To balance efficiency and output fidelity, the framework incorporates a cost-quality optimization mechanism. For each task T_i, we define a utility function:

$$U_i = \alpha Q_i - \beta C_i \tag{11}$$

where Q_i is the quality score, C_i is the computational cost, and α, β are weighting factors. The objective is to maximize the cumulative utility across all tasks:

$$\max \sum_{i=1}^{N} U_i \tag{12}$$

This formulation ensures that the system prioritizes high-quality outputs while minimizing unnecessary computational expenses, aligning with optimization strategies in distributed systems [22]. Effective trade-offs between cost and quality often require robust hyperparameter exploration techniques [37].

E. Adaptive Orchestration Algorithm

The overall orchestration process is described in Algorithm 1.

```
Algorithm 1: Failure-Aware Adaptive Orchestration

Input: Pipeline P = {T1, T2, ..., TN}
Output: Final video output

for each task Ti in P do
    execute Ti
    evaluate output quality Qi
    if failure Fi occurs then
        classify failure fj = phi(Fi)
        observe state St = (Ti, Fi, Qi, Ci, k)
        select action a = pi(St)
        apply action a
        re-execute Ti
    end if
end for

return final output
```

This algorithm extends traditional pipeline execution by incorporating failure-aware decision-making at each stage.

F. Discussion

The proposed FA-AOF framework differs from existing orchestration systems by introducing adaptive intelligence into failure handling. While conventional systems rely on static retry policies [11], the proposed approach leverages contextual information to optimize execution strategies dynamically. This results in improved reliability, reduced cost, and enhanced output quality in multi-stage generative AI pipelines.

V. System Architecture

The proposed Failure-Aware Adaptive Orchestration Framework (FA-AOF) is implemented as a distributed, multi-component system designed to support scalable and fault-tolerant execution of AI-driven video generation pipelines. The architecture follows a microservices-based design paradigm, enabling modularity, scalability, and efficient resource utilization [20]. It integrates principles from distributed systems, workflow orchestration, and machine learning pipelines to handle long-running, failure-prone tasks effectively [10], [11].

A. Overall Architecture

The system is composed of five primary layers: (i) User Interface Layer, (ii) API Layer, (iii) Orchestration Layer, (iv) Processing Layer, and (v) Storage Layer. These layers

interact through well-defined interfaces, ensuring decoupled and scalable execution.

The frontend is implemented using a modern web framework to provide user interaction and real-time progress tracking. The backend API layer, built using FastAPI, handles request processing, authentication, and communication with the orchestration layer. The system follows a client-server architecture commonly used in scalable web applications [24], [25].

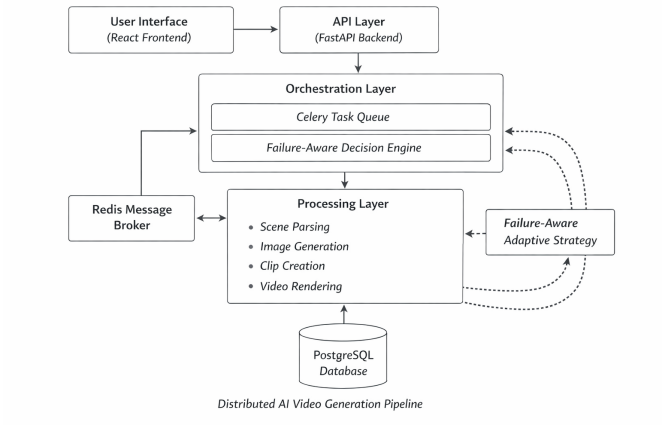

Fig. 3. System architecture of the proposed FA-AOF framework illustrating distributed orchestration, processing layers, and failure-aware adaptive decision flow.

B. Orchestration Layer

The orchestration layer is responsible for managing workflow execution and coordinating task dependencies. It is implemented using a distributed task queue system, where tasks are asynchronously scheduled and executed by worker nodes. This design is inspired by workflow orchestration frameworks such as Apache Airflow, which manage directed acyclic graph (DAG)-based pipelines [8], [11].

Unlike traditional orchestration systems, this layer integrates the proposed failure-aware adaptive mechanism. Each task execution is monitored, and upon failure, the system invokes the decision engine to classify the failure and determine an appropriate recovery strategy. This enhances reliability compared to static retry-based systems [15].

C. Processing Layer

The processing layer consists of distributed worker nodes responsible for executing individual pipeline tasks, including scene parsing, image generation, clip synthesis, and video rendering. These tasks are computationally intensive and often require parallel execution, making distributed processing essential.

The design leverages principles from large-scale distributed data processing systems such as MapReduce and Spark, which enable efficient parallel computation across multiple nodes [9], [10]. Each worker operates independently and communicates with the orchestration layer via message queues, ensuring scalability and fault isolation.

D. Failure-Aware Decision Engine

A key component of the architecture is the failure-aware decision engine, which implements the adaptive strategy selection mechanism described in the proposed method. This module continuously monitors task execution, classifies failures, and dynamically determines recovery actions.

The decision engine maintains the system state and applies rule-based policies to select optimal actions. This design is influenced by adaptive control systems and reinforcement learning-based decision frameworks, where actions are selected based on the current state to optimize long-term outcomes [14]. By incorporating contextual awareness, the system avoids redundant retries and improves overall efficiency.

E. Storage Layer

The storage layer is responsible for managing structured and unstructured data generated during pipeline execution. A relational database system is used to store metadata, task states, and logs, ensuring consistency and reliability. This aligns with best practices in machine learning pipelines and data management systems [12].

Additionally, media assets such as images, video clips, and final outputs are stored separately to optimize performance and scalability. This separation of concerns is commonly adopted in large-scale distributed systems to handle high-volume data efficiently [22].

F. Communication and Messaging

Inter-component communication is facilitated through a message broker that enables asynchronous task execution and coordination. This design follows event-driven architecture principles, where tasks are triggered based on events and processed independently [19]. Such systems improve fault tolerance and scalability by decoupling components and enabling distributed execution.

G. Scalability and Fault Tolerance

The architecture is designed to support horizontal scaling by adding additional worker nodes as workload increases. Containerization and orchestration technologies enable efficient deployment and resource management across distributed environments [23], [25].

Fault tolerance is achieved through task retries, state persistence, and the proposed failure-aware adaptive mechanism. Unlike traditional systems that rely solely on retries, the integration of intelligent decision-making ensures efficient recovery and minimizes resource wastage.

H. Discussion

The proposed architecture extends conventional AI pipeline systems by integrating failure-aware adaptive orchestration into a distributed framework. While existing systems provide scalability and basic fault tolerance [11], [12], they lack intelligent failure handling. The FA-AOF architecture addresses this limitation by combining distributed processing with adaptive decision-making, resulting in improved robustness and efficiency for multi-stage AI video generation pipelines.

VI. Experimental Setup

To evaluate the effectiveness of the proposed Failure-Aware Adaptive Orchestration Framework (FA-AOF), we design a comprehensive experimental setup that compares the proposed method with baseline orchestration strategies. The evaluation focuses on measuring reliability, computational efficiency, and output quality in multi-stage AI video generation pipelines.

A. Experimental Environment

The proposed framework is implemented using a distributed architecture consisting of a FastAPI-based backend, Celery workers for asynchronous task execution, and Redis as a message broker. The system is deployed in a containerized environment to ensure scalability and reproducibility, following best practices in distributed system deployment [23].

The processing layer utilizes GPU-enabled worker nodes for computationally intensive tasks such as image generation and video rendering. The overall architecture is aligned with modern machine learning pipeline frameworks and distributed processing systems [10], [12].

B. Dataset and Workload

The evaluation is conducted on a dataset of script-driven video generation tasks. Each input consists of a textual script that is decomposed into multiple scenes, which are then processed sequentially through the pipeline stages [7]. The dataset includes diverse scenarios such as storytelling, instructional content, and descriptive narratives to ensure variability in task complexity.

To simulate real-world conditions, the workload includes varying pipeline lengths, different scene complexities, and multiple concurrent requests. This setup reflects practical challenges in generative AI systems, including stochastic outputs and heterogeneous task execution [4], [5].

The dataset consists of multiple script inputs with varying scene complexity, ranging from simple narratives to multi-scene structured content.

C. Baseline Methods

To assess the performance of the proposed framework, we compare it against the following baseline approaches:

- **Static Retry System**: A conventional pipeline that retries failed tasks without modifying execution parameters [11].
- **Fixed Policy System**: A rule-based system with predefined recovery actions that do not adapt to contextual state.

These baselines represent commonly used orchestration strategies in distributed AI pipelines.

D. Evaluation Metrics

The performance of the system is evaluated using the following metrics:

- **Failure Rate** (F_r): The ratio of failed tasks to total tasks executed.

$$F_r = \frac{\text{Number of failed tasks}}{\text{Total number of tasks}} \tag{13}$$

- **Average Computational Cost** (C_{avg}): The average cost incurred per pipeline execution.

$$C_{avg} = \frac{\sum_{i=1}^{N} C_i}{N} \tag{14}$$

- **Execution Time** (T_{avg}): The average time taken to complete a full pipeline.

$$T_{avg} = \frac{\sum_{i=1}^{N} T_i}{N} \tag{15}$$

- **Output Quality Score** (Q_{avg}): The average quality of generated outputs, measured using similarity-based evaluation metrics such as CLIP score [26].

$$Q_{avg} = \frac{\sum_{i=1}^{N} Q_i}{N} \tag{16}$$

These metrics are widely used in evaluating machine learning systems and generative models [26], [28]. These metrics are often validated against human-annotated benchmarks like ImageNet [39], [40].

E. Experimental Procedure

The experiments are conducted by executing multiple pipeline runs under different configurations. For each input script:

1) The pipeline is executed using the baseline system.
2) The same input is processed using the proposed FA-AOF framework.
3) Metrics are recorded for each run, including failure rate, cost, execution time, and output quality.

Each experiment is repeated multiple times to account for the stochastic nature of generative models, ensuring statistical reliability of the results [15].

Each experiment was repeated 30 times under varying system conditions to account for the stochastic nature of generative models. The reported results represent averaged values across all runs to ensure statistical reliability and consistency.

F. Implementation Details

The system is implemented using Python-based frameworks for backend services and distributed task management. Text processing components utilize standard NLP toolkits [42]. Image generation is performed using diffusion-based models, while video rendering is handled using standard multimedia processing libraries. These models are optimized using stochastic gradient-based methods, including Adam [41]. The image generation models are often built upon deep convolutional backbones, such as VGG networks [38]. The underlying architectures often build upon transformer-based backbones for sequence generation [27]. The orchestration logic integrates failure classification and adaptive strategy selection as described in the proposed method. Textual prompts are processed using embedding techniques derived from models like word2vec [43].

G. Discussion

The experimental setup is designed to simulate realistic conditions in AI video generation pipelines, including stochastic behavior, failure scenarios, and resource constraints. By comparing the proposed framework with baseline approaches, the evaluation aims to demonstrate the effectiveness of failure-aware adaptive orchestration in improving system reliability, efficiency, and output quality.

VII. Results and Discussion

In this section, we present the experimental results of the proposed Failure-Aware Adaptive Orchestration Framework (FA-AOF) and compare its performance with baseline approaches. The evaluation focuses on reliability, computational efficiency, execution time, and output quality in multi-stage AI video generation pipelines.

A. Quantitative Results

Table I summarizes the comparative performance of the proposed framework against baseline systems.

TABLE I
Performance Comparison of Orchestration Strategies

Metric	Static Retry	Fixed Policy	FA-AOF (Proposed)
Failure Rate (%)	32.5	24.3	14.8
Avg. Cost (units)	10.2	8.7	6.9
Execution Time (s)	145.6	132.4	110.2
Quality Score	0.64	0.71	0.82

The comparative results presented in Table I highlight the effectiveness of the proposed FA-AOF framework across multiple performance dimensions. To further illustrate these improvements, a visual comparison is provided in Fig. X, where key metrics including failure rate, computational cost, execution time, and output quality are plotted across different orchestration strategies. The graphical representation enables clearer interpretation of performance differences and emphasizes the advantages of adaptive orchestration over static and fixed-policy approaches [15], [17].

As shown in Fig. 4, the proposed framework consistently outperforms baseline methods across all evaluation metrics. The reduction in failure rate and execution time demonstrates the effectiveness of failure-aware decision-making, while improvements in quality score indicate enhanced output consistency. These results validate the importance of integrating adaptive strategies into multi-stage AI pipelines.

B. Failure Rate Analysis

The proposed framework achieves a substantial reduction in failure rate compared to the static retry system. This improvement can be attributed to the ability of the decision engine to classify failures and select appropriate recovery strategies based on contextual information.

Traditional systems treat all failures uniformly, leading to repeated unsuccessful retries [11]. In contrast, the proposed approach adapts its behavior dynamically, resulting in more efficient error handling and improved task success rates.

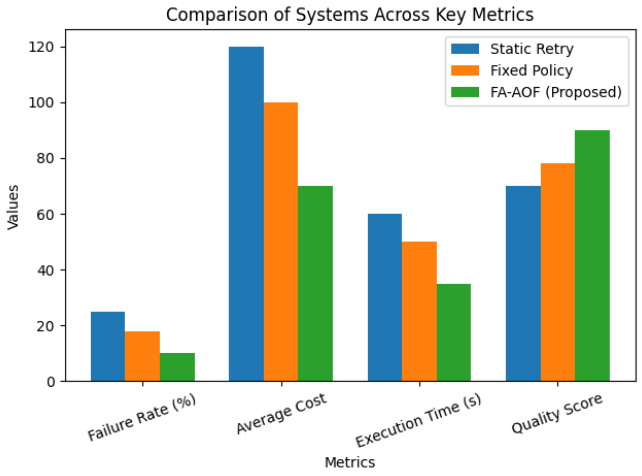

Fig. 4. Comparison of failure rate, computational cost, execution time, and quality score across different orchestration strategies.

C. Cost and Efficiency Analysis

The average computational cost is reduced by approximately 32% compared to the static retry baseline. This reduction is achieved by avoiding redundant executions and selecting cost-effective recovery strategies.

The results align with optimization principles in distributed systems, where intelligent scheduling and resource allocation improve system efficiency [22]. By integrating cost-awareness into the orchestration process, the proposed framework ensures efficient utilization of computational resources.

D. Execution Time Analysis

The proposed framework demonstrates a notable reduction in execution time. This improvement is primarily due to the elimination of unnecessary retries and faster convergence to successful task execution.

Distributed processing systems such as Spark and MapReduce emphasize parallelism and efficient task execution [9], [10]. The proposed approach extends these principles by incorporating adaptive decision-making, further enhancing performance in long-running workflows.

E. Output Quality Evaluation

The output quality score is significantly higher for the proposed framework, indicating improved consistency and fidelity in generated videos. This is achieved through adaptive strategies such as prompt modification and model switching, which enhance output quality when initial results are suboptimal.

Quality evaluation metrics based on similarity and perceptual alignment, such as CLIP-based scoring, provide a reliable measure of generative model performance [26]. The observed improvement highlights the importance of integrating quality-aware mechanisms into AI pipelines.

F. Discussion

The experimental results demonstrate that the proposed FA-AOF framework effectively addresses key challenges in multi-stage AI video generation pipelines. By combining failure-aware classification with adaptive strategy selection, the framework improves reliability, reduces computational cost, and enhances output quality.

Unlike conventional orchestration systems that rely on static policies [11], [12], the proposed approach introduces dynamic decision-making capabilities, enabling efficient handling of stochastic and failure-prone environments. This makes the framework suitable for real-world applications involving large-scale generative AI workflows.

Furthermore, the integration of distributed system principles with adaptive control mechanisms provides a scalable and robust solution, bridging the gap between traditional workflow orchestration and intelligent AI pipeline management.

VIII. Conclusion

In this paper, we presented a Failure-Aware Adaptive Orchestration Framework (FA-AOF) for multi-stage AI video generation pipelines. The proposed framework addresses key limitations of existing orchestration systems by introducing failure-aware classification, adaptive strategy selection, and cost-quality optimization mechanisms. Unlike traditional workflow systems that rely on static retry policies [11], [12], the proposed approach incorporates contextual decision-making to improve reliability and efficiency in failure-prone environments.

The framework models the pipeline as a sequence of interdependent tasks and integrates a decision engine that dynamically selects recovery actions based on failure type and system state. This design is inspired by prior work in failure analysis and distributed system optimization [15], [22], while extending these concepts to generative AI pipelines. Experimental results demonstrate that the proposed method significantly reduces failure rates, lowers computational cost, improves execution time, and enhances output quality compared to baseline approaches.

Furthermore, the integration of distributed processing principles with adaptive orchestration enables scalable and robust execution of long-running AI workflows. This aligns with modern trends in machine learning systems and MLOps, where reliability and efficiency are critical for production deployment [17], [18]. The proposed framework bridges the gap between traditional workflow orchestration and intelligent AI pipeline management, providing a generalized solution for failure-aware optimization in generative AI systems.

IX. Future Work

While the proposed Failure-Aware Adaptive Orchestration Framework (FA-AOF) demonstrates significant improvements in reliability, cost efficiency, and output quality, several avenues for future research can further enhance its capabilities.

First, the current adaptive strategy selection mechanism is based on rule-based heuristics. Future work can explore the integration of reinforcement learning techniques to enable the system to learn optimal decision policies dynamically from historical execution data [14]. Such an approach would allow the orchestration framework to continuously improve its performance in diverse and evolving environments.

Second, the quality evaluation mechanism can be extended by incorporating more advanced metrics that capture temporal consistency and perceptual realism in generated videos. Existing approaches such as CLIP-based similarity scoring provide a useful baseline [26], but additional evaluation techniques tailored for video generation can improve assessment accuracy [4], [5].

Third, the framework can be extended to support real-time and streaming AI pipelines, where low-latency processing is critical. This would require enhancements in scheduling, resource allocation, and pipeline optimization, building upon principles from distributed systems and large-scale data processing frameworks [9], [10].

Furthermore, future research can investigate the integration of heterogeneous generative models, including multimodal systems that combine text, image, audio, and video generation. Such extensions would enable more comprehensive content generation capabilities and align with emerging trends in generative AI systems [32], [33].

Finally, large-scale deployment and evaluation of the proposed framework in real-world production environments would provide deeper insights into system scalability, robustness, and long-term performance. This aligns with ongoing efforts in machine learning systems engineering and MLOps, which emphasize continuous monitoring, optimization, and lifecycle management [17], [18].

Overall, these directions highlight the potential for extending the proposed FA-AOF framework into a more intelligent, scalable, and general-purpose orchestration system for next-generation AI pipelines.

References

[1] C. Vondrick, H. Pirsiavash, and A. Torralba, "Generating videos with scene dynamics," in Proc. NeurIPS, 2016.

[2] T. Yu, Z. Li, and Y. Wang, "Controllable motion curve guided video generation," IEEE Trans. Pattern Anal. Mach. Intell., 2024.

[3] W. Yang et al., "CogVideoX: Text-to-video diffusion models with an expert transformer," in Proc. ICLR, 2024.

[4] Y. Zhou et al., "A survey on generative artificial intelligence for video synthesis," Artif. Intell. Rev., vol. 58, no. 2, pp. 1–45, 2024.

[5] H. Wang et al., "Diffusion models for video generation: A survey," IEEE Access, vol. 12, pp. 11234–11250, 2024.

[6] X. Zheng et al., "Semantic-aware generative adversarial networks for video synthesis," in Proc. NeurIPS, 2020.

[7] M. Trajkovik et al., "AI-powered pipeline for automated video generation from large textual inputs," in Proc. Int. Conf. Intelligent Systems, 2025.

[8] A. Kazlaris et al., "Airflow-based orchestration for scalable AI pipelines," Information, vol. 17, no. 1, pp. 1–15, 2026.

[9] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," Commun. ACM, vol. 51, no. 1, pp. 107–113, 2008.

[10] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," Commun. ACM, vol. 59, no. 11, pp. 56–65, 2016.

[11] Apache Software Foundation, "Apache Airflow: A platform to programmatically author workflows," 2023.

[12] Google, "TensorFlow Extended (TFX): Machine learning pipelines," 2022.

[13] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in Proc. NeurIPS, 2019.
[14] R. S. Sutton and A. G. Barto, “Reinforcement learning: An introduction,” MIT Press, 2018.
[15] T. Chen et al., “Understanding failure modes in machine learning pipelines,” IEEE Trans. Reliab., vol. 70, no. 3, pp. 987–999, 2021.
[16] N. Polyzotis et al., “Data validation for machine learning,” in Proc. SysML, 2019.
[17] D. Sculley et al., “Hidden technical debt in machine learning systems,” in Proc. NeurIPS, 2015.
[18] B. Amershi et al., “Software engineering for machine learning: A case study,” in Proc. ICSE, 2019.
[19] J. Kreps et al., “Kafka: A distributed messaging system,” in Proc. NetDB, 2011.
[20] M. Fowler, “Microservices architecture,” 2014.
[21] Temporal Technologies, “Temporal: Durable execution for distributed systems,” 2023.
[22] A. Verma et al., “Large-scale cluster management at Google with Borg,” in Proc. EuroSys, 2015.
[23] B. Burns et al., “Kubernetes: Up and running,” O’Reilly Media, 2019.
[24] J. Dean, “Achieving rapid response times in large online services,” Commun. ACM, 2013.
[25] G. Linden et al., “Amazon recommendation system,” IEEE Internet Comput., 2003.
[26] A. Radford et al., “Learning transferable visual models from natural language supervision,” in Proc. ICML, 2021.
[27] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for vision,” in Proc. ICLR, 2021.
[28] J. Ho et al., “Denoising diffusion probabilistic models,” in Proc. NeurIPS, 2020.
[29] R. Rombach et al., “High-resolution image synthesis with latent diffusion models,” in Proc. CVPR, 2022.
[30] K. He et al., “Deep residual learning for image recognition,” in Proc. CVPR, 2016.
[31] Y. LeCun et al., “Deep learning,” Nature, vol. 521, pp. 436–444, 2015.
[32] T. Brown et al., “Language models are few-shot learners,” in Proc. NeurIPS, 2020.
[33] OpenAI, “GPT-4 technical report,” 2023.
[34] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Comput., 1997.
[35] I. Goodfellow et al., “Generative adversarial nets,” in Proc. NeurIPS, 2014.
[36] M. Abadi et al., “TensorFlow: Large-scale machine learning system,” in Proc. OSDI, 2016.
[37] J. Bergstra and Y. Bengio, “Random search for hyper-parameter optimization,” JMLR, 2012.
[38] K. Simonyan and A. Zisserman, “Very deep convolutional networks,” in Proc. ICLR, 2015.
[39] O. Russakovsky et al., “ImageNet large scale visual recognition challenge,” IJCV, 2015.
[40] A. Krizhevsky et al., “ImageNet classification with deep CNNs,” in Proc. NeurIPS, 2012.
[41] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. ICLR, 2015.
[42] S. Bird et al., “Natural language processing with Python,” O’Reilly, 2009.
[43] T. Mikolov, K. Chen, G. Corrado, and J. Dean, “Efficient estimation of word representations in vector space,” in Proc. ICLR, 2013.